

Multi-Agent Mamba-RL for Efficient Sequential Recommendation

Zhi-Quan Feng

Abstract

Sequential recommendation is naturally connected to reinforcement learning because a recommender repeatedly observes user behavior, selects items, and receives feedback. In practice, however, reinforcement-learning recommenders face three persistent difficulties: user interests evolve at different time scales, logged datasets provide only partial counterfactual evidence, and large language or sequence backbones are expensive to execute inside every policy update. This report presents the independent Multi-Agent Mamba-RL pipeline implemented in `MediaTek_2026_project`. The pipeline uses a frozen Mamba-2.8B model once to produce reusable item-text embeddings, then trains three lightweight agents over those embeddings: a long-term preference agent, a short-term intent agent, and a coordinator agent. Each agent owns a disjoint low-rank adaptation module. The two specialists use Mamba-inspired selective state-space recurrences with different observation horizons and decay time scales, while the coordinator learns user-dependent mixing weights and produces the final ranking policy.

Training follows a deliberately staged procedure: supervised specialist warm-up, frozen-specialist coordinator warm-up, and joint REINFORCE optimization. The implementation supports MovieLens, Amazon Reviews 2023, and local Yelp 2019/2023 snapshots; performs chronological leave-two-out splitting; evaluates every user against the complete item catalog; records ranking and throughput metrics; and generates a grounded natural-language reason after recommendation. The architecture separates expensive semantic encoding, high-frequency policy learning, catalog ranking, and explanation generation. This separation makes the system suitable as a reproducible research baseline for studying multi-agent specialization, selective state dynamics, parameter-efficient policy learning, and high-throughput recommendation.

1 Introduction

Recommender systems have traditionally been formulated as rating prediction, click prediction, or next-item ranking problems. These formulations are effective when the objective is immediate relevance, but they do not fully represent the interactive nature of recommendation. A recommendation changes what the user sees, the resulting response changes the observable user state, and that new state influences subsequent recommendations. Reinforcement learning (RL) offers a natural language for this process by representing the recommender as a policy, the user as an environment, and engagement or satisfaction as a reward [3].

The promise of RL does not make its application straightforward. Industrial recommenders operate over thousands or millions of discrete actions, whereas most deep RL algorithms were developed for much smaller action spaces. Real user exploration is costly, so policies are usually trained from logged data generated by an earlier recommender. Feedback is incomplete: a missing click does not prove that an unseen item was irrelevant. A reward such as click-through rate may also favor immediate attraction even when the intended objective is long-term satisfaction, retention, or diversity. These issues make policy design, reward design, and offline evaluation inseparable parts of an RL recommender.

Sequential user representation creates another challenge. A user can exhibit a stable preference over months while simultaneously pursuing a short-lived intent within the current session. A single recurrent state must decide how quickly to forget old evidence, yet one global time scale is unlikely to fit both behaviors. Transformer recommenders can model long dependencies, but their self-attention cost

grows quadratically with sequence length. Selective state-space models such as Mamba instead update a recurrent state in linear time and make the update input-dependent [2]. This mechanism is attractive for recommendation because an interaction can be selectively retained or forgotten according to its content.

A direct combination of a large Mamba language model and RL would nevertheless be computationally inefficient. If three agents each executed an independent 2.8-billion-parameter model for every sampled action, policy learning would be dominated by repeated text encoding rather than recommendation. Storing three complete adapted backbones would also undermine the parameter-efficiency motivation. The central systems question is therefore not simply whether Mamba can replace a Transformer, but how its semantic and selective-state capabilities can be used without placing a large foundation model inside the high-frequency optimization loop.

This project studies a fundamental question: *Can a shared Mamba semantic space and several low-rank, time-specialized policy agents provide an efficient and interpretable foundation for reinforcement-learning-based sequential recommendation?*

The proposed answer is a three-agent architecture. A frozen Mamba model first converts item metadata into a cached semantic matrix. A long-term agent and a short-term agent process sequences of these vectors with different selective state dynamics. A coordinator observes both specialist states, learns when each one should dominate, and produces the final catalog ranking. The agents are first initialized through supervised next-item learning and are then jointly optimized using policy gradients. This pipeline is implemented independently of the existing deep-learning and single-policy RL baselines, so it can be evaluated without changing their behavior.

2 Related Works and Innovations

2.1 Reinforcement Learning for Recommendation

Early deep RL recommenders commonly formulated recommendation as a Markov decision process and learned action values or policies from user feedback. DRN used deep Q-learning for news recommendation and explicitly modeled future reward and user return behavior [4]. List-wise and page-wise methods expanded the action from a single item to a slate, but this introduced a combinatorial action space. SlateQ addressed this problem by decomposing slate value under assumptions about user choice [6]. Policy-gradient methods offered another route: the Top- K off-policy REINFORCE recommender scaled policy learning to a production catalog and corrected bias from logged behavior policies [5].

These studies establish the reasons to use RL—exploration, delayed effects, and long-term value—but they also expose a gap between research formulations and common offline experiments. Many sequential recommenders use a sampled next-item RL objective even though the evaluation contains no environment transition caused by the recommended action. Recent work showed that part of the apparent benefit can arise because the RL term acts as an auxiliary representation-learning objective rather than because it learns a genuinely long-horizon policy [7]. A credible implementation must therefore distinguish its current offline objective from claims about online long-term satisfaction.

The present pipeline adopts a conservative interpretation. Its implemented reward is a sampled next-item reward, so it is best understood as a cooperative policy-gradient sequential ranker or contextual-bandit baseline. The code is structured so that the same agent interface can later receive simulator or online rewards, but the current MovieLens, Amazon, and Yelp evaluation does not claim to reconstruct unobserved counterfactual behavior.

2.2 Mamba and Sequential Recommendation

Transformer-based sequential recommenders capture rich dependencies but require quadratic attention computation. Mamba introduces input-dependent state-space parameters and linear sequence processing [2]. Mamba4Rec applied selective state-space modeling to sequential recommendation [8], while

SIGMA introduced bidirectional and gated mechanisms to improve contextual and short-term modeling [9]. These results show that Mamba is useful for sequential recommendation, but replacing a Transformer with a standard Mamba block does not solve RL credit assignment, preference time-scale separation, or foundation-model cost.

This project uses Mamba in two distinct roles. The frozen Mamba-2.8B language model provides item semantics from natural-language metadata. Lightweight Mamba-inspired diagonal selective recurrences then provide trainable policy states. This separation preserves linear policy updates without executing the language model at every transition.

The three agents are therefore not three copies of Mamba-2.8B. They share one cached item-semantic space and own separate low-rank updates inside compact selective-state policies. Independent adaptation is retained while memory and computation remain proportional to the small recommendation dimension.

2.3 Multi-Agent and Multi-Time-Scale Recommendation

A multi-agent recommender should contain more than several neural modules. Each agent needs a distinct observation, policy, action distribution, reward signal, or functional role. In the proposed architecture, the long-term agent observes up to the full configured history, the short-term agent observes a recent window, and both define separate categorical policies. The coordinator is a third policy that observes specialist states and produces the deployed distribution.

This division reflects the fact that stable preferences and transient intents evolve at different rates. A user may consistently prefer one genre while temporarily searching within another category. A short-term specialist can react quickly, whereas a slowly changing specialist can resist noisy or accidental interactions. The coordinator performs context-dependent arbitration instead of using a fixed average.

LoRA represents an update with two low-rank matrices while leaving the base path unchanged [1]. Giving each agent its own LoRA parameters creates separate adaptation subspaces at a small cost. Zero-initializing the second matrix makes every adapter begin as a no-op and stabilizes staged training.

2.4 Why This Topic Must Be Investigated

Scientifically, the topic tests whether multi-agent specialization is more than extra parameter count. If long- and short-term agents receive identical inputs and objectives, they may collapse to nearly identical policies. A useful architecture must create structural differences through observation horizon, state time scale, and independent policy feedback, then verify these choices through ablation.

The topic also clarifies what Mamba contributes. A pretrained Mamba can supply textual knowledge, but repeatedly invoking it inside RL may erase any throughput advantage. A tiny recurrent model is efficient but may fail to exploit item metadata. Separating semantic precomputation from selective policy-state updates provides a testable middle ground.

Catalog ranking must also remain practical. Training over the full catalog at every step is expensive, while sampled-negative evaluation can exaggerate quality. This pipeline trains on small slates but evaluates by matrix multiplication against the complete catalog. It reports users per second and user-item scores per second so that efficiency is measurable.

Finally, explanation generation should not dominate ranking latency. The pipeline performs retrieval first and loads the generator only after evaluation. This creates a clear boundary between the policy and its post-hoc verbalization.

2.5 Innovations and Contributions

The implementation makes five contributions:

1. A cooperative three-agent recommender with structurally different long- and short-term selective-

state specialists and a learned coordinator.

2. Disjoint LoRA parameters for every agent with shared cached Mamba semantics, avoiding three copies or forward passes of the 2.8B backbone.
3. Three-stage optimization separating specialist learning, coordinator calibration, and joint policy-gradient adaptation.
4. A unified MovieLens, Amazon Reviews 2023, and Yelp 2019/2023 evaluation interface with chronological splitting and full-catalog ranking.
5. Explanation generation isolated from ranking, with learned agent weights included in qualitative output.

3 Method

3.1 Problem Formulation

Let $\mathcal{U}$ be users and $\mathcal{I}$ the catalog, with $N = |\mathcal{I}|$. User interactions are ordered:

$$\mathcal{H}_u = (i_1, i_2, \dots, i_{T_u}). \quad (1)$$

At transition t , the state is prefix $s_t = (i_1, \dots, i_t)$ and logged next item i_{t+1} is positive. The policies are

$$\pi_L(a \mid s_L), \quad \pi_S(a \mid s_S), \quad \pi_C(a \mid s_L, s_S). \quad (2)$$

3.2 Pipeline Overview

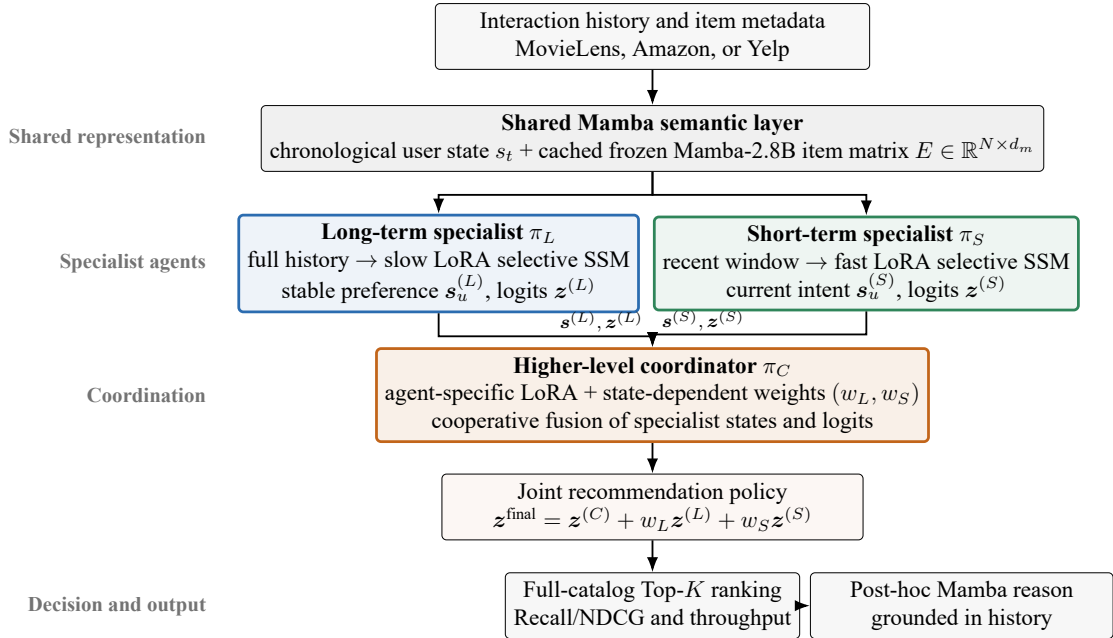


Figure 1: Hierarchical cooperative Multi-Agent Mamba-RL architecture. Solid arrows show the inference path; dashed red arrows show training-only policy feedback.

The hierarchy assigns complementary responsibilities rather than making three agents vote independently. The two specialist policies extract preferences at different time scales, and the higher-level coordinator decides their state-dependent contribution to one deployed recommendation policy. During joint training, the coordinator and both specialists receive policy-gradient signals; during inference, only the solid forward path is executed.

3.3 Data and Chronological Splitting

Every record becomes (u, i, τ, t_i) : user, item, timestamp, and item text. IDs become contiguous integers. Users need at least five positive events by default. MovieLens and Yelp use rating at least four.

Table 1: Dataset interfaces and item text.

Argument	Interactions	Item text
movielens	ratings	title and genres
amazon-<category>	Amazon Reviews 2023	title, category, features, description
amazon:<exact-config>	exact HF config	review/meta join by <code>parent_asin</code>
yelp19, yelp23	local Yelp JSON/table	name, categories, location, attributes
synthetic	deterministic events	short names for tests

Yelp is license-gated and requires a local path. Leave-two-out splitting is

$$\mathcal{H}_u^{\text{train}} = (i_1, \dots, i_{T_u-2}), \quad (3)$$

$$i_u^{\text{valid}} = i_{T_u-1}, \quad (4)$$

$$i_u^{\text{test}} = i_{T_u}. \quad (5)$$

Training prefixes produce $((i_1, \dots, i_k), i_{k+1})$. Reservoir sampling caps transitions at `MAX_TRANSITIONS`.

3.4 Cached Mamba Semantics

Frozen Mamba-2.8B encodes metadata:

$$e_i^{\text{text}} = \text{MeanPool}(\text{Mamba}_{\text{frozen}}(t_i)) \in \mathbb{R}^{d_m}. \quad (6)$$

A dataset/item/text fingerprint controls cache reuse. A shared bridge gives

$$e_i = \text{norm}(W_p e_i^{\text{text}}), \quad W_p \in \mathbb{R}^{d \times d_m}, \quad d = 128. \quad (7)$$

3.5 Agent-Specific LoRA

For input x , agent j owns

$$\text{LoRA}_j(x) = \frac{\alpha}{r} B_j A_j \text{Dropout}(x). \quad (8)$$

Defaults are $r = 8$, $\alpha = 16$, and dropout 0.05. A_j uses Kaiming initialization and B_j starts at zero. Specialists own candidate, delta, gate, and output adapters; the coordinator owns state, mixture, and score adapters.

3.6 Selective-State Specialists

For $j \in \{L, S\}$:

$$\delta_t^{(j)} = \text{softplus} \left(b_\delta^{(j)} + \text{LoRA}_\delta^{(j)}(x_t) \right), \quad (9)$$

$$a_t^{(j)} = \exp(-\delta_t^{(j)}), \quad (10)$$

$$\tilde{h}_t^{(j)} = \tanh \left(x_t + \text{LoRA}_{\text{candidate}}^{(j)}(x_t) \right), \quad (11)$$

$$h_t^{(j)} = a_t^{(j)} \odot h_{t-1}^{(j)} + (1 - a_t^{(j)}) \odot \tilde{h}_t^{(j)}, \quad (12)$$

$$g_t^{(j)} = \text{sigmoid} \left(\text{LoRA}_{\text{gate}}^{(j)}(x_t) \right), \quad (13)$$

$$o_t^{(j)} = g_t^{(j)} \odot h_t^{(j)} + (1 - g_t^{(j)}) \odot x_t. \quad (14)$$

Small δ_t preserves history; large values adopt the current item. Long sees up to 100 events at initial time scale 0.08; short sees 10 at 0.5. Logits are

$$z_i^{(j)} = \exp(\tau_j) \langle \mathbf{s}_u^{(j)}, \mathbf{e}_i \rangle. \quad (15)$$

3.7 Coordinator

$$\mathbf{q}_u = [\mathbf{s}_u^{(L)}; \mathbf{s}_u^{(S)}], \quad (16)$$

$$[w_L, w_S] = \text{softmax}(\text{LoRA}_{\text{mix}}(\mathbf{q}_u)), \quad (17)$$

$$\mathbf{s}_u^{(C)} = \text{norm} \left(w_L \mathbf{s}_u^{(L)} + w_S \mathbf{s}_u^{(S)} + \text{LoRA}_{\text{state}}(\mathbf{q}_u) \right), \quad (18)$$

$$z_i^{\text{final}} = z_i^{(C)} + w_L z_i^{(L)} + w_S z_i^{(S)}. \quad (19)$$

4 Training Strategy

4.1 Stage 1: Specialist Warm-Up

A slate is $\mathcal{C} = \{i^+, i_1^-, \dots, i_{K-1}^-\}$ with $K = 64$. Coordinator is frozen:

$$\mathcal{L}_{\text{specialist}} = \text{CE}(\mathbf{z}^{(L)}, 0) + \text{CE}(\mathbf{z}^{(S)}, 0). \quad (20)$$

4.2 Stage 2: Coordinator Warm-Up

Specialists and projection are frozen:

$$\mathcal{L}_{\text{coord}} = \text{CE}(\mathbf{z}^{\text{final}}, 0). \quad (21)$$

4.3 Stage 3: Joint REINFORCE

Each policy forms $\pi_j(a | s) = \text{softmax}(\mathbf{z}^{(j)})_a$ and samples an action:

$$r_j = \begin{cases} 1, & a_j = 0, \\ -0.05, & a_j \neq 0. \end{cases} \quad (22)$$

Baselines use $b_j \leftarrow 0.95b_j + 0.05\bar{r}_j$ and $A_j = r_j - b_j$. The policy loss is

$$\mathcal{L}_{\text{PG}} = - \sum_j \mathbb{E}[A_j \log \pi_j(a_j | s)] - 0.01 \sum_j \mathbb{E}[H(\pi_j)]. \quad (23)$$

Advantage is detached. Additional terms are

$$\mathcal{L}_{\text{sup}} = \frac{1}{3} \sum_{j \in \{L, S, C\}} \text{CE}(\mathbf{z}^{(j)}, 0), \quad (24)$$

$$\mathcal{L}_{\text{spec}} = \cos^2(\mathbf{s}_u^{(L)}, \mathbf{s}_u^{(S)}), \quad (25)$$

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{PG}} + 0.1\mathcal{L}_{\text{sup}} + 0.01\mathcal{L}_{\text{spec}}. \quad (26)$$

4.4 Gradients and Optimization

Coordinator gradients flow through fusion into both specialist LoRA paths. Each specialist also receives its own policy loss. This is cooperative local credit rather than a centralized counterfactual critic.

Table 2: Default optimization.

Stage	Learning rate	Trainable components
Specialists	2×10^{-4}	long, short, projection
Coordinator	2×10^{-4}	coordinator
Joint RL	5×10^{-5}	all components

AdamW uses weight decay 10^{-5} . CUDA uses FP16 autocast, GradScaler, and gradient clipping at 1.0.

5 Full-Catalog Evaluation and Throughput

Training uses sampled slates, but evaluation projects the catalog once:

$$E = W_p E_{\text{text}} \in \mathbb{R}^{N \times d}, \quad Z = S E^\top \in \mathbb{R}^{B \times N}. \quad (27)$$

This GEMM avoids a $B \times N \times d$ expansion. Seen non-target items receive $-\infty$.

$$\text{rank}_u = \sum_{i=1}^N \mathbb{I}[z_{u,i} \geq z_{u,i+}], \quad (28)$$

$$\text{Recall@K} = \frac{1}{|\mathcal{U}|} \sum_u \mathbb{I}[\text{rank}_u \leq K], \quad (29)$$

$$\text{NDCG@K} = \frac{1}{|\mathcal{U}|} \sum_u \frac{\mathbb{I}[\text{rank}_u \leq K]}{\log_2(\text{rank}_u + 1)}. \quad (30)$$

The code reports $K = 5, 10$, users/s, and scores/s.

Throughput comes from cached Mamba vectors, shared semantics, linear recurrences, capped history, sampled training candidates, FP16, bounded transitions, GEMM ranking, and delayed generator loading.

6 Recommendation Explanation

For sampled test users, Top-1, ten recent item texts, and coordinator weights form:

Explain this recommendation in one concise sentence using only the supplied history.
 Long-term agent weight=<wL>; short-term agent weight=<wS>.
 History: <recent item texts>.
 Recommended item: <recommended item text>. Reason:

Mamba greedily generates at most 40 tokens after ranking. The explanation is grounded but post-hoc; it is not a causal proof and does not affect reward.

7 Experimental Design

Required ablations include single versus three agents, identical versus 100/10 windows, fixed versus learned fusion, direct joint RL versus staged training, and removal of specialization or supervised anchoring. Semantic features should compare Mamba vectors, random vectors, and ID embeddings. Sequence baselines should include GRU and Transformer under matched capacity. Random negatives should be compared with hard negatives.

Beyond Recall/NDCG, useful measures include coverage, novelty, diversity, results by history length, coordinator-weight distribution, specialist-state cosine similarity, transitions/s, users/s, scores/s, peak GPU memory, and explanation latency.

8 Implementation and Reproducibility

Table 3: Independent implementation files.

File	Responsibility
run_mamba_rl.sh	launcher
src/data_mamba_rl.py	dataset adapters
src/model_mamba_rl.py	agents and LoRA
src/train_mamba_rl.py	training, evaluation, explanations
tests/test_mamba_rl.py	tests
outputs_mamba_rl/	artifacts

```
bash run_mamba_rl.sh movielens-1m 0
bash run_mamba_rl.sh movielens-20m 0
bash run_mamba_rl.sh amazon-all-beauty 0
bash run_mamba_rl.sh amazon-movies-and-tv 0
bash run_mamba_rl.sh yelp19 0 /data/yelp-2019
bash run_mamba_rl.sh yelp23 0 /data/yelp-2023
```

Environment variables include `CACHE_DIR`, `BATCH_SIZE`, `EVAL_BATCH_SIZE`, `SPECIALIST_EPOCHS`, `COORDINATOR_EPOCHS`, `RL_EPOCHS`, `MAX_TRANSITIONS`, and `SEED`. A smoke test is

```
python -m src.train_mamba_rl \
  --dataset synthetic --device cpu --skip-mamba \
  --no-generate-reasons --max-transitions 32 --batch-size 8
```

Outputs are `train.log`, `multi_agent_lora.pt`, `loss.png`, `metrics.json`, and recommendations under `outputs_mamba_rl/<dataset>/<run-id>/`. Legacy training paths do not import this pipeline.

9 Limitations

1. The $+1/-0.05$ next-item reward is not online feedback from the learned policy.
2. Static logs contain no action-dependent user transition, so this is not a complete long-horizon MDP.
3. Random negatives may be unexposed; no propensity correction is used.
4. Credit assignment uses local rewards rather than a centralized critic.

5. Training uses sampled slates while evaluation uses the full catalog.
6. Explanations are post-hoc and not optimized for faithfulness.
7. Mamba-2.8B supplies semantics; agents use compact Mamba-inspired recurrences rather than three full 2.8B adapters.

Recall/NDCG gains therefore mean improved offline ranking, not proven long-term satisfaction.

10 Future Research

Reward-adaptive memory can use

$$\delta_t = f_\theta(\mathbf{x}_t, r_{t-1}, \sigma_{t-1}). \quad (31)$$

Offline RL can add propensity weighting or

$$\mathcal{L}_{\text{CQL}} = \log \sum_a \exp Q(s, a) - Q(s, a_{\text{logged}}). \quad (32)$$

A simulator can support centralized critics and retention reward. Item selection and reasoning can be jointly optimized with

$$r = \lambda_1 r_{\text{rank}} + \lambda_2 r_{\text{grounding}} + \lambda_3 r_{\text{consistency}} + \lambda_4 r_{\text{diversity}}. \quad (33)$$

11 Conclusion

The pipeline combines Mamba item semantics, linear selective-state modeling, agent-specific LoRA, and policy gradients without placing three large models in the RL loop. Mamba-2.8B establishes a reusable semantic catalog; long and short agents model different time scales; and a coordinator learns context-dependent fusion. Staged training establishes specialists, calibrates coordination, and performs joint adaptation. Full-catalog evaluation and isolated explanation generation make ranking quality and systems cost measurable.

References

- [1] E. Hu, Y. Shen, P. Wallis, et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.
- [2] A. Gu and T. Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *COLM*, 2024.
- [3] M. M. Afsar, T. Crump, and B. Far. Reinforcement Learning Based Recommender Systems: A Survey. *ACM Computing Surveys*, 2022.
- [4] G. Zheng, F. Zhang, Z. Zheng, et al. DRN: A Deep Reinforcement Learning Framework for News Recommendation. *The Web Conference*, 2018.
- [5] M. Chen, A. Beutel, P. Covington, et al. Top-K Off-Policy Correction for a REINFORCE Recommender System. *WSDM*, 2019.
- [6] E. Ie, V. Jain, J. Wang, et al. Reinforcement Learning for Slate-Based Recommender Systems. *arXiv:1905.12767*, 2019.
- [7] A. L. Silva, D. Parra, and R. T. Icarte. On the Unexpected Effectiveness of Reinforcement Learning for Sequential Recommendation. *ICML*, 2024.

- [8] C. Liu, J. Lin, J. Wang, et al. Mamba4Rec: Towards Efficient Sequential Recommendation with Selective State Space Models. *arXiv:2403.03900*, 2024.
- [9] Z. Liu, Q. Liu, Y. Wang, et al. SIGMA: Selective Gated Mamba for Sequential Recommendation. *AAAI*, 2025.